Module 3: Portfolio Milestone

Start Assignment

- Points 50
- Submitting a file upload

Portfolio Milestone (50 points)

Important! Read First

Choose one of the following two assignments to complete this week. Do not do both assignments. Identify your assignment choice in the title of your submission. Review the **Module 8 Portfolio Project (https://csuglobal.instructure.com/courses/123286/assignments/2269095)** instructions.

Option #1: Using C Programming language

In your Linux installation, modify the numbers.sh script, to do the following:

1. Add to the script commands to display the system time, before and after the process. Optional: store the system time in variables, to display just the difference at the end of the process, i.e. how many hours/minutes/seconds it took to run this script. Hint: use the built-in **SECONDS variable (https://www.oreilly.com/library/view/shell-scripting-expert/9781118166321/c03-anchor-3.xhtml)** to do most of the work.
2. Modify the “for” loop to repeat one million (1,000,000) times.
3. Save the script and delete file1.txt that was created from the previous exercise.
4. Run the script.

You should now have a file called file1.txt containing one million lines, with each line being a random number. You should also have information indicating how long this process took to run.

Create a C program to perform this task, to create file2.txt, and compare execution times. Can you use what you have learned in the last 2 modules, like multithreading or synchronization, to make this process run faster? How would you do that? Use at least 2 other methods to try to improve execution time.

Write down the summary of the results, with the following information:

1. Describe the different methods you used to perform this task, with the times each one took.

2. Explain why you chose each of those methods.
3. Did each of these methods perform as you expected them to, or were there any surprises? Describe your findings in detail.
4. If your system had double its current processing power (CPU power), how much of an improvement would you expect for this process? Explain the reasons for that estimate, and provide references to support your opinion.

Option #2: Using Python Programming language

In your Linux installation, modify the numbers.sh script, to do the following:

1. Add to the script commands to display the system time, before and after the process. Optional: store the system time in variables, to display just the difference at the end of the process, i.e. how many hours/minutes/seconds it took to run this script. Hint: use the built-in **SECONDS variable (https://www.oreilly.com/library/view/shell-scripting-expert/9781118166321/c03-anchor-3.xhtml)** to do most of the work.
2. Modify the “for” loop to repeat one million (1,000,000) times.
3. Save the script and delete file1.txt that was created from the previous exercise.
4. Run the script.

You should now have a file called file1.txt containing one million lines, with each line being a random number. You should also have information indicating how long this process took to run.

Create a Python program to perform this task, to create file2.txt, and compare execution times. Can you use what you've learned in the last 2 modules, like multithreading or synchronization, to make this process run faster? How would you do that? Use at least 2 other methods to try to improve execution time.

Write down the summary of the results, with the following information:

1. Describe the different methods you used to perform this task, with the times each one took.
2. Explain why you chose each of those methods.
3. Did each of these methods perform as you expected them to, or were there any surprises? Describe your findings in detail.
4. If your system had double its current processing power (CPU power), how much of an improvement would you expect for this process? Explain the reasons for that estimate, and provide references to support your opinion.

Criteria	Ratings				Pts
Script Modification & Execution	**10 pts** **Meets Expectation** Script correctly modified to display system time before and after execution; SECONDS variable or equivalent used effectively; loop successfully runs 1,000,000 times; file1.txt correctly generated with random numbers.	**8 pts** **Approaches Expectation** Minor syntax or logic errors in timing or loop implementation; script runs but may produce partial or incorrect output.	**6 pts** **Below Expectation** Major issues prevent full execution or correct timing output; random number generation inconsistent.	**4 pts** **Limited Evidence** Little to no modification or script fails to run.	10 pts
C or Python Program Functionality	**10 pts** **Meets Expectation** Program successfully replicates numbers.sh functionality; generates file2.txt correctly; execution time captured and displayed accurately.	**8 pts** **Approaches Expectation** Program mostly works but may have minor issues with output, file generation, or timing accuracy.	**6 pts** **Below Expectation** Program compiles/runs but produces incomplete or incorrect results; timing not implemented properly.	**4 pts** **Limited Evidence** Program missing, not functional, or unrelated to assignment.	10 pts
Performance Optimization Methods	**10 pts** **Meets Expectation** At least two optimization methods (e.g., multithreading, synchronization, buffering, I/O optimization) are clearly	**8 pts** **Approaches Expectation** Two methods attempted but not fully implemented or explained; limited discussion of performance impact.	**6 pts** **Below Expectation** Only one method used, or methods applied incorrectly; minimal or no improvement observed.	**4 pts** **Limited Evidence** No optimization methods used or discussed.	10 pts

Criteria	Ratings				Pts
	implemented and described; improvements are measurable and explained.				
Comparative Analysis of Execution Times	**8 pts** **Meets Expectation** Clear, well-organized table or discussion comparing execution times across methods (Bash, C/Python, optimizations); analysis highlights performance differences with reasoning.	**6 pts** **Approaches Expectation** Execution times presented but comparison lacks depth or clarity.	**4 pts** **Below Expectation** Execution time data incomplete, inconsistent, or lacks analysis.	**2 pts** **Limited Evidence** No execution time comparison or explanation provided.	8 pts
Reflection and System Power Analysis	**7 pts** **Meets Expectation** Insightful reflection on results; unexpected outcomes discussed thoughtfully; CPU power doubling scenario explained logically with evidence and references supporting reasoning.	**5.25 pts** **Approaches Expectation** Adequate reflection; some reasoning for CPU power analysis but limited support or depth.	**3.5 pts** **Below Expectation** Minimal reflection or unclear explanation of CPU impact; lacks references or sound reasoning.	**1.75 pts** **Limited Evidence** No reflection or analysis of results; missing CPU power discussion.	7 pts
Clarity, Organization, and Professional	**5 pts** **Meets Expectation** Submission is clear, well-	**3.75 pts** **Approaches Expectation** Mostly clear but may have some	**2.5 pts** **Below Expectation** Disorganized or confusing	**1.25 pts** **Limited Evidence** Submission incomplete,	5 pts

Criteria	Ratings				Pts
Presentation	structured, and professional; methods and results are easy to follow; language is concise and technically accurate.	organizational or language issues; minor formatting or clarity problems.	presentation; multiple errors reduce readability.	unclear, or unprofessional.	
					Total Points: 50